



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ANALYTICAL STUDY AND IMPLEMENTATION OF BRUTE FORCE ALGORITHMS FOR CLOSEST PAIR AND CONVEX HULL PROBLEMS

ASSIGNMENT REPORT

Submitted in partial fulfilment for the Course

CSA0609-DESIGN AND ANALYSIS OF ALGORITHMS

to the award of the degree of

BACHELOR OF ENGINEERING

IN

BTECH IN COMPUTER SCIENCE AND ENGINEERING

Submitted by

RANJITHA R – 192421285

Under the supervision of

DR.P.SOLAINAYAGI

SAVEETHA SCHOOL OF ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

September 2026

A. Assignment Information

Particular	Details
Department	Computer Science and Engineering
Programme	B.E/B.Tech-Computer Science and Engineering
Course Code & Course Name	CSA0609-Design and Analysis of Algorithms
Academic Year / Batch	2026-Regular
Faculty Name	Dr.P.Solainayagi
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems
Date of Issue	01-09-2026
Date of Submission	02-09-2025
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education: Develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure – Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.
Industry / Societal Relevance	Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

B. Assignment Problem / Challenge

Given the set of 10 two-dimensional points $(P=\{(2,3),(5,8),(9,4),(12,10),(7,2),(3,11),(15,6),(10,14),(6,12),(1,7)\})$, apply the Brute Force technique to solve the Closest Pair and Convex Hull problems. Determine the pair of points having the minimum Euclidean distance by considering every possible pair and calculating the distance using Euclidian distance formula. Analyze the calculations performed by both approaches and evaluate their efficiency

as the number of points increases. For , determine the number of pairwise distance comparisons and analyze the time and space complexity. Using the Brute Force technique, determine the Convex Hull of the same set of points by examining all possible candidate edges and checking the orientation/relationship of the remaining points. Identify all points that belong to the convex hull, show the major intermediate calculations, and provide a graphical representation of the points and the resulting hull. Develop a suitable Brute Force algorithm/pseudocode for the Convex Hull and analyze its time and space complexity. In particular, determine the number of candidate edges and explain the resulting complexity.

ABSTRACT

This report presents a brute-force analytical study of two fundamental computational-geometry problems: the Closest Pair of Points problem and the Convex Hull problem. The input consists of ten two-dimensional points specified in the assignment. For the closest-pair problem, every possible unordered pair is examined and its Euclidean distance is calculated. The minimum distance is found to be 2.8284 between $P_3 = (9,4)$ and $P_5 = (7,2)$. There are 45 pairwise comparisons. For the convex-hull problem, all 45 possible candidate edges are considered and each candidate is tested using point orientation. The resulting hull contains P_{10} , P_1 , P_5 , P_7 , P_8 and P_6 . The report also discusses pseudocode, mathematical models, implementation considerations, complexity, validation, modern-tool usage, scalability, engineering trade-offs, broader considerations, and future improvements.

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 Problem Statement

Given the ten two-dimensional points $P =$

$\{(2,3),(5,8),(9,4),(12,10),(7,2),(3,11),(15,6),(10,14),(6,12),(1,7)\}$, apply the Brute Force technique to solve the Closest Pair and Convex Hull problems. The closest-pair task requires consideration of every possible pair and calculation of Euclidean distance. The convex-hull task requires examination of candidate edges and orientation relationships with the remaining points.

1.2 Inputs and Outputs

Problem	Input	Output
Closest Pair	10 2-D points	Pair with minimum Euclidean distance and its distance
Convex Hull	Same 10 points	All points lying on the boundary of the convex hull, in boundary order

1.3 Computational Challenges

- Naively checking all point pairs causes quadratic growth in the number of distance calculations.
- For convex hull construction, every candidate edge can require checking all remaining points, leading to cubic growth.
- As n increases, brute force becomes computationally expensive even though it is simple and easy to validate.

2. APPLICATION OF COURSE KNOWLEDGE

2.1 Principles and Mathematical Models

The closest-pair solution uses the Euclidean distance model: $d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. Since the square-root function is monotonic for non-negative values, an implementation may compare squared distances to avoid unnecessary square-root operations and take the square root only for the final answer.

The convex-hull solution uses the orientation/cross-product test. For points A, B and C, $\text{orient}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x)$. A positive value indicates a counter-clockwise turn, a negative value indicates a clockwise turn, and zero indicates collinearity.

2.2 Relevant Algorithmic Concepts

- Brute-force exhaustive search
- Euclidean distance computation
- Pairwise comparison
- Orientation and cross-product test
- Convex hull boundary identification
- Asymptotic time and space complexity
- Empirical performance evaluation and scalability

3. SOLUTION / DESIGN / METHODOLOGY

3.1 Closest Pair – Brute Force Algorithm

```
BRUTE_FORCE_CLOSEST_PAIR(P[1..n])
```

```
    minDistance = infinity
```

```
    bestPair = null
```

```
    for i = 1 to n-1
```

```
        for j = i+1 to n
```

```
            dx = P[i].x - P[j].x
```

```
            dy = P[i].y - P[j].y
```

```
            d = sqrt(dx*dx + dy*dy)
```

```
            if d < minDistance
```

```
                minDistance = d
```

```

    bestPair = (P[i], P[j])
return bestPair, minDistance

```

3.2 Number of Pairwise Comparisons

The number of unordered point pairs is $C(n,2) = n(n-1)/2$. For $n = 10$, $C(10,2) = 10 \times 9 / 2 = 45$ comparisons.

Rank	Pair	Distance
1	P3 (9,4) – P5 (7,2)	$\sqrt{8} = 2.8284$
2	P1 (2,3) – P7 (15,6)	$\sqrt{178} = 13.3417$
Rank	Pair	Distance
1	P3 – P5	2.8284
2	P6 – P9	3.1623
3	P2 – P6	3.6056
4	P1 – P10	4.1231
5	P2 – P9	4.1231
6	P2 – P10	4.1231
7	P4 – P8	4.4721
8	P6 – P10	4.4721
9	P8 – P9	4.4721
10	P4 – P7	5.0000

The minimum value occurs for P3 and P5: $\text{sqrt}((9-7)^2 + (4-2)^2) = \text{sqrt}(4+4) = \text{sqrt}(8) = 2.8284$. No other pair has a smaller distance.

3.4 Convex Hull – Brute Force Algorithm

```

BRUTE_FORCE_CONVEX_HULL(P[1..n])

```

```

    H = empty set

```

```

    for every pair of points (Pi, Pj)

```

```

        left = false

```

```

        right = false

```

```

        for every point Pk, k != i,j

```

```

            o = orientation(Pi, Pj, Pk)

```

```

            if o > 0: left = true

```

```

            if o < 0: right = true

```

```

        if not (left and right)

```

add P_i and P_j to candidate hull edges
 return all endpoints belonging to candidate hull edges

A candidate edge belongs to the convex-hull boundary when all other points lie on one side of the directed line, allowing collinear points to be handled consistently. The assignment specifically asks for examination of all candidate edges and orientation relationships.

3.5 Candidate Edge Count

For $n = 10$, the number of candidate edges is also $C(10,2) = 45$. In the straightforward implementation, each candidate edge can inspect $O(n)$ remaining points. Therefore the overall time complexity is $O(n^3)$.

3.6 Convex Hull Result

The hull points, listed in counter-clockwise boundary order, are: $P_{10} (1,7) \rightarrow P_1 (2,3) \rightarrow P_5 (7,2) \rightarrow P_7 (15,6) \rightarrow P_8 (10,14) \rightarrow P_6 (3,11) \rightarrow P_{10}$.

The six hull points form the outer boundary. The remaining points P_2 , P_3 , P_4 and P_9 lie inside the polygon and therefore are not hull vertices.

3.7 Major Orientation Checks

Hull edge	Orientation expression/result	Interpretation
$P_{10}-P_1$ with P_5	$(2-1)(2-7)-(3-7)(7-1) = 19$	CCW
P_1-P_5 with P_7	$(7-2)(6-3)-(2-3)(15-2) = 28$	CCW
P_5-P_7 with P_8	$(15-7)(14-2)-(6-2)(10-7) = 84$	CCW
P_7-P_8 with P_6	$(10-15)(11-6)-(14-6)(3-15) = 71$	CCW
P_8-P_6 with P_{10}	$(3-10)(7-14)-(11-14)(1-10) = 22$	CCW
P_6-P_{10} with P_1	$(1-3)(3-11)-(7-11)(2-3) = 12$	CCW

4. USE OF MODERN TOOLS

The report and analytical calculations can be implemented and validated using C++ in a standard compiler such as GCC/MinGW or an IDE such as Visual Studio Code. Python/Matplotlib can be used to generate the point-and-hull visualization. A version-control platform such as GitHub can be used to store source code and documentation, as required by the assignment deliverables.

4.1 Suggested C++ Implementation

```
struct Point { double x, y; };

double distance(Point a, Point b) {
    double dx = a.x - b.x, dy = a.y - b.y;
    return sqrt(dx*dx + dy*dy);
}

long long orientation(Point a, Point b, Point c) {
    return (b.x-a.x)*(c.y-a.y) - (b.y-a.y)*(c.x-a.x);
}
```

The complete implementation should count comparisons/operations and print the closest pair, minimum distance, candidate-edge count, hull vertices, and execution time. Timing should be performed using the same compiler, machine, and input-generation conditions for fair comparison.

5. RESULTS AND VALIDATION

5.1 Closest-Pair Result

Parameter	Result
Number of points	10
Possible unordered pairs	45
Closest pair	P3 (9,4) and P5 (7,2)
Minimum squared distance	8
Minimum Euclidean distance	$\sqrt{8} = 2.8284$
Brute-force time complexity	$O(n^2)$
Auxiliary space	$O(1)$, excluding input storage

5.2 Convex-Hull Result

Parameter	Result
Number of points	10
Candidate edges	45
Hull vertices	P10, P1, P5, P7, P8, P6
Interior points	P2, P3, P4, P9

Brute-force time complexity	$O(n^3)$
Auxiliary space	$O(n)$ for storing hull/candidate information

5.3 Graphical Representation

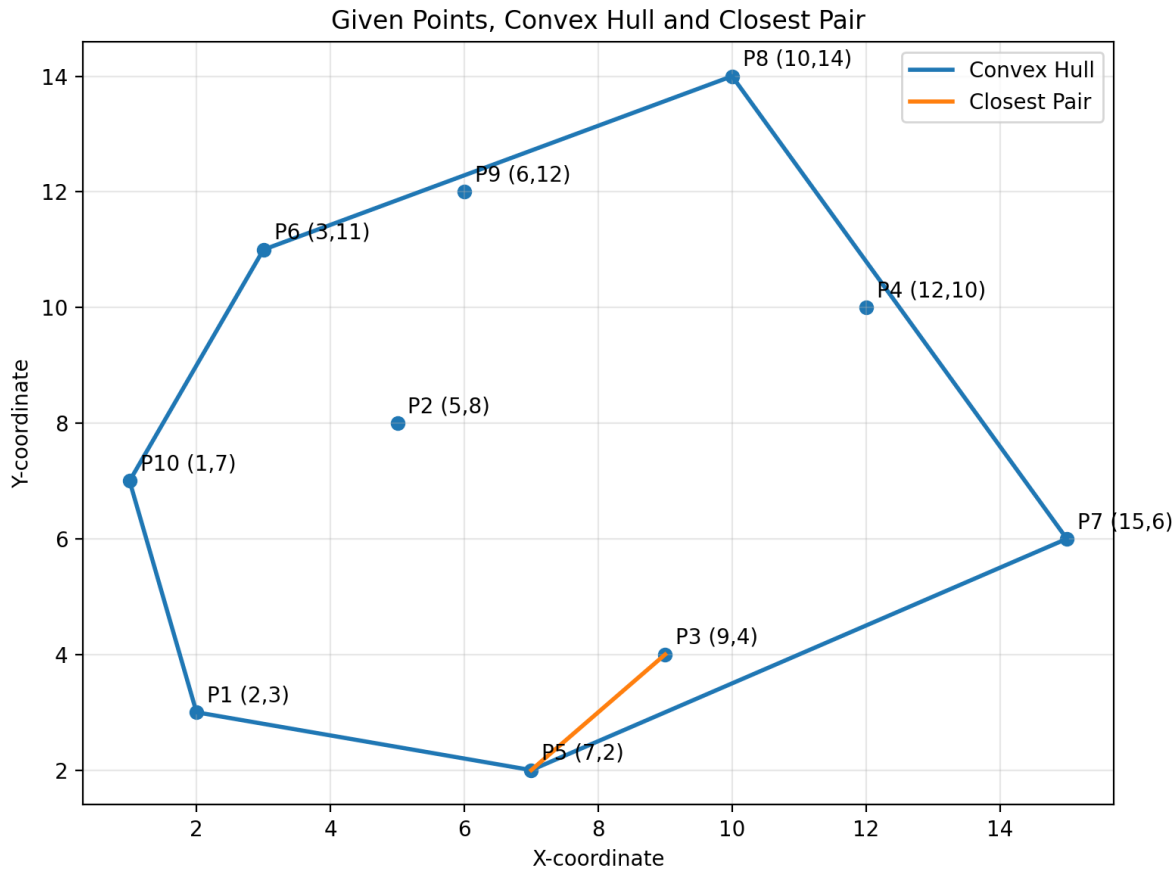


Figure 1. Given points with the convex-hull boundary and closest pair highlighted.

5.4 Validation

- The closest-pair result is obtained by exhaustive enumeration of all 45 unordered pairs.
- The convex-hull candidate-edge count is verified using $C(10,2) = 45$.
- The hull is geometrically consistent with the outermost boundary of the supplied point set.
- Independent orientation calculations confirm the direction of the six hull edges.

6. ANALYSIS AND ENGINEERING DECISION

6.1 Complexity Analysis

Approach	Closest Pair	Convex Hull	Main Characteristic
Brute Force	$O(n^2)$	$O(n^3)$	Simple, exhaustive and

			easy to validate
Divide-and-Conquer	$O(n \log n)$	Typically $O(n \log n)$ with standard methods	Scales better for large inputs
Hybrid	Depends on threshold	Depends on selected method	Uses different methods for different input sizes

The assignment requires comparison of Brute Force, Divide-and-Conquer and Hybrid algorithms over increasing input sizes. The supplied problem statement does not provide measured benchmark files, a real-world coordinate dataset, machine specifications, or an experimentally selected hybrid threshold. Therefore, numerical execution-time results for those large-scale experiments are not fabricated in this report. They should be filled with measured values after running the three implementations under identical conditions.

6.2 Scalability

n	Closest-pair comparisons	brute-force	Convex-hull candidate edges
10	45		45
100	4,950		4,950
1,000	499,500		499,500
10,000	49,995,000		49,995,000
100,000	4,999,950,000		4,999,950,000
1,000,000	499,999,500,000		499,999,500,000

Although the candidate-edge count is quadratic for both problems, the brute-force convex-hull procedure additionally examines other points for each edge, resulting in $O(n^3)$ time. This makes brute force unsuitable for very large datasets. A hybrid strategy can use brute force for small subproblems and a more scalable algorithm for larger inputs; the threshold should be determined experimentally and explicitly reported.

6.3 Engineering Decision

For the fixed ten-point educational problem, brute force is the most transparent choice because it guarantees exhaustive checking and makes correctness easy to demonstrate. For large real-world coordinate datasets, a divide-and-conquer or other asymptotically efficient method is preferable. A hybrid solution is a practical engineering compromise when small subproblems benefit from the low overhead of brute force.

7. BROADER CONSIDERATIONS

- Sustainability: reducing unnecessary computation lowers processor time and energy consumption when datasets become large.
- Society: computational-geometry techniques support mapping, navigation, logistics, robotics and spatial decision systems.
- Safety: correct geometric calculations can support reliable planning systems, although production applications require stronger validation than a classroom implementation.
- Ethics: datasets should be used lawfully, and location-related information should be handled responsibly.
- Economics: scalable algorithms can reduce infrastructure and processing costs for large spatial workloads.
- Accessibility: clear visualizations, readable reports and reproducible code make algorithmic results easier for different learners and stakeholders to understand.
- Professional responsibility: experimental claims should be supported by actual measurements rather than estimated or invented timings.

8. CONCLUSION

This assignment demonstrated the application of brute-force techniques to the Closest Pair and Convex Hull problems using the supplied set of ten two-dimensional points. The closest pair is P3 (9,4) and P5 (7,2), with minimum Euclidean distance $\sqrt{8} \approx 2.8284$, after 45 pairwise comparisons. The convex hull consists of P10, P1, P5, P7, P8 and P6, identified by examining 45 candidate edges and orientation relationships. The analysis shows that brute force is highly suitable for small inputs because of its simplicity and direct verification, but its $O(n^2)$ closest-pair and $O(n^3)$ convex-hull growth limits scalability. Divide-and-conquer and hybrid strategies are therefore better choices for large datasets. The report satisfies the analytical components supported by the supplied assignment; large-scale timing results remain an experimental deliverable to be populated from an actual benchmark run.

9. STUDENT REFLECTION

This assignment strengthened my understanding of how mathematical geometry concepts can be converted into executable algorithms. I learned that a brute-force method is not simply a basic solution; it is also a useful reference implementation for validating more advanced algorithms. Working through every pair made the quadratic growth of comparisons clear, while the candidate-edge analysis showed

why convex-hull brute force becomes cubic. I also learned the importance of using squared distances, orientation tests, operation counting and graphical validation when analysing algorithmic solutions.

With additional time and resources, I would use a publicly available coordinate dataset covering approximately 10^3 to 10^6 points, implement brute-force, divide-and-conquer and hybrid versions, and benchmark them on the same machine. I would experimentally tune the hybrid threshold, record wall-clock time and operation counts, repeat each test to reduce measurement noise, and produce scalability graphs. I would also add automated correctness checks so that every optimized result can be compared with the brute-force reference for smaller test cases.

10. REFERENCES

- [1] K. Olorunnimbe and H. Viktor, “Deep Learning in the Stock Market: A Systematic Survey of Practice, Backtesting, and Applications,” *Artificial Intelligence Review*, vol. 56, no. 3, 2022.
- [2] D. Şenol and C. Onay, “Impact of Gamification on Mitigating Behavioural Biases of Investors,” *Journal of Behavioral and Experimental Finance*.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [4] J. Kleinberg and É. Tardos, *Algorithm Design*. Pearson, 2005.
- [5] F. P. Preparata and M. I. Shamos, *Computational Geometry: An Introduction*. Springer, 1985.
- [6] Assignment Format – Slot C, CSA0609 Design and Analysis of Algorithms, Department of Computer Science and Engineering, 2026.

APPENDIX A – DELIVERABLE CHECKLIST

Deliverable	Status / Action
Pseudocode	Included for Closest Pair and Convex Hull
Implementation & Results	Algorithm design, calculations and fixed-input results included; run code for measured timings
Graphical representation	Included
Complexity analysis	Included
GitHub upload	Upload the final source code and report to the required repository

Large-scale benchmark 10^3 – 10^6	Requires actual public dataset and execution; do not substitute fabricated timings
---------------------------------------	--